



THE IMAGINATION UNIVERSITY PROGRAMME

RVfpga Lab 10

Serial Buses

Custom Edition

1. INTRODUCTION

In this lab, we first describe how serial buses work and the main features of one of the most typical serial buses currently used, the SPI bus (Section 2). We then focus on the SPI OLED available on the Nexys Video board: we analyse the high-level specification for this peripheral and propose fundamental exercises (Sections 3 and 4), and then we analyse its low-level implementation and propose some advanced exercises (Sections 5 and 6).

2. SERIAL BUSES – THE SPI BUS

Parallel buses send several bits at once, whereas serial buses send one bit at a time. We first compare these two communication schemes and then describe the SPI (serial peripheral interface) protocol, which is one of the most common serial buses currently used. You can find lots of information on the internet for extending your knowledge about this important communication protocol.

As already demonstrated in previous labs, the main purpose of embedded electronics is to connect processors and circuits to create desired functions. In order for processors and circuits to share information, they must share a common communication protocol. Hundreds of communication protocols have been defined to achieve this data exchange, and, in general, they can be separated into two main categories: parallel or serial interfaces.

Parallel interfaces transfer multiple bits in parallel, i.e., at the same time. They require buses (multiple wires) of data. For example, the protocol may transmit eight, sixteen, or more bits at the same time (see Figure 1). They also require a clock to time when new groups of N data bits are ready to transfer.

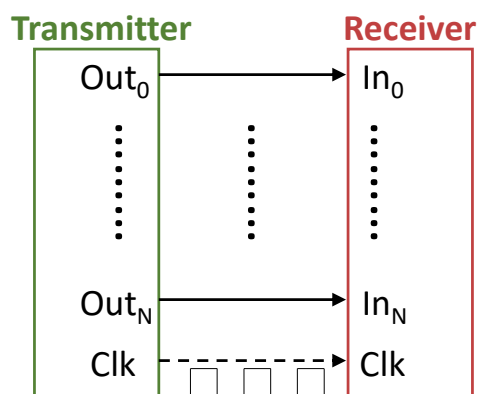


Figure 1. Example of a parallel 8-bit data bus.

In contrast to parallel communication, serial interfaces stream their data one bit at a time. These interfaces can operate using as few as one wire and usually never more than four. Figure 2 shows an example serial interface with one wire for data and one for a clock. At each new clock edge, a new data bit is transferred.



Figure 2. Example of a serial 1-bit data bus.

Parallel communication has the benefits of being fast, straightforward, and relatively easy to implement. However, it requires many more input/output (I/O) lines. So, because pins are limited, embedded systems often opt for serial communication, sacrificing potential speed for pin real estate.

SPI Bus:

The Serial Peripheral Interface (SPI) protocol is one of the most widely used interfaces between microcontroller and peripheral ICs such as sensors, ADCs, DACs, shift registers, SRAM, and others. SPI is a synchronous, full duplex interface based on controller-peripheral (formerly called master-slave) communication.

The SPI bus usually communicates via 4 ports (see Figure 3):

- **SDO** – Serial Data Out: Controller's output to peripheral device
- **SDI** – Serial Data Input: Controller's input from peripheral device
- **SCK** – Serial Clock: Sent from controller to peripheral device
- **CS** – Chip Select: Active low signal; Controller sends signal (0 when peripheral is selected) to peripheral

Note: historically, SDO has also been called MOSI (master data out, slave data in) and SDI has been called MISO (master data in, slave data out). Those terms are outdated and offensive, but they still exist in the literature and in documentation.

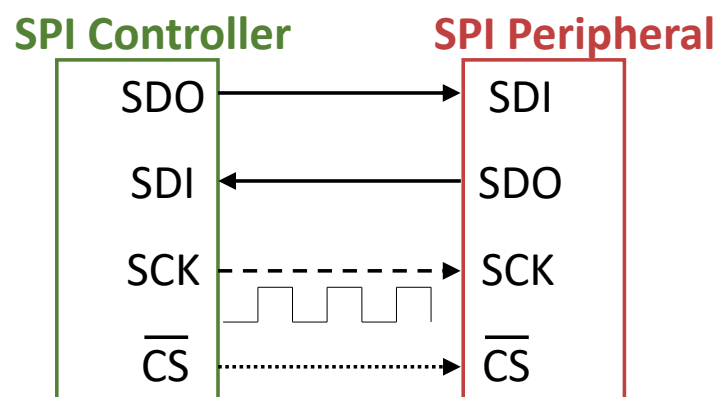


Figure 3. Example of a system with one SPI controller and one SPI peripheral.

The serial data is synchronized to the rising or falling clock edge. SPI is a full-duplex interface; the controller and the peripheral can send data at the same time via the SDO and SDI lines, respectively. SPI interfaces only have one controller, but they may have multiple peripherals. When more than one peripheral is connected, multiple low-asserted chip select signals ($\overline{\text{CS}}$) from the controller are used to select which peripheral is being accessed. SDO and SDI are the serial data lines: SDO (serial data out) is the output data from the controller to the peripheral and SDI (serial data in) is the input data from the peripheral to the controller.

To initiate SPI communication, the controller must select the peripheral device (by asserting the $\overline{\text{CS}}$ signal, i.e., $\overline{\text{CS}} = 0$) and then sending the clock signal to the peripheral. During SPI communication, the data is simultaneously transmitted from and to the controller through the SDO and SDI signals, respectively. The serial clock (SCK) edge synchronizes the sampling of the data.

The SPI interface also provides additional signals, CPOL and CPHA, for selecting the idle state of the clock and the phase for sampling the signal. The clock polarity (CPOL) signal is 0 when the clock (SCK) idles at 0 and 1 when it idles at 1. The clock phase (CPHA) signal selects the phase of the clock to send and sample data. When CPHA = 0, data (on SDI or SDO) is sampled on the leading edge (i.e., the first edge after SCK stops idling - and on every cycle thereafter); so data (SDI and SDO) must change on the trailing edge, as shown in the top two timing diagrams of Figure 4. CPHA = 1 does the opposite: data is sampled on the trailing edge and data changes on the leading edge, as shown in the bottom two figures of Figure 4. The edge on which new data is transmitted is also called the *shifting edge*, because this serial communication is typically implemented using a shift register.

The SPI interface we use in this lab is CPHA = 0 and CPOL = 0, so SCK idles low and the controller and peripheral sample data on the rising edge and shift new data onto the line (SDO or SDI) just after each falling edge, as shown in the top timing diagram of Figure 4. Note that when SCK is idle, and just before it rises, SDO and SDI must carry the most significant bit of the next data byte.

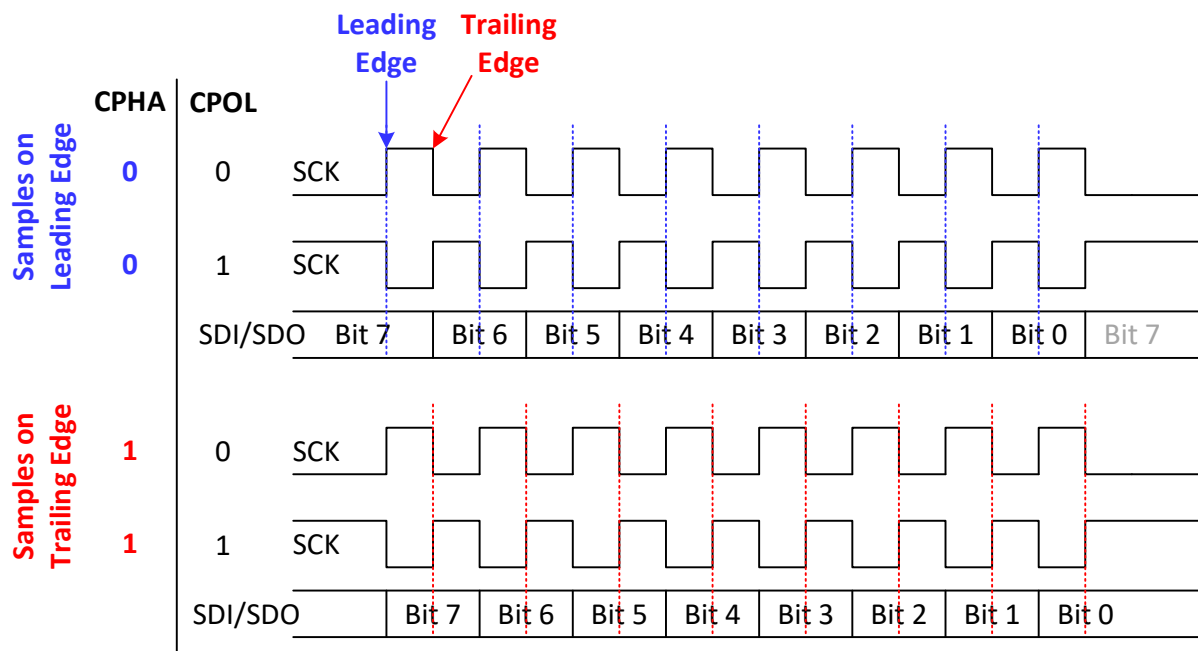


Figure 4. Relationship of CPHA/CPOL with sampling/sending data

3. SPI OLED: HIGH-LEVEL SPECIFICATION

Many peripherals include an SPI interface. For example, the OLED on the Nexys Video board has an SPI interface. In this section we describe the high-level specification of the RVfpga System's SPI controller and introduce the UG-2832HSWEG04 OLED included on the Nexys Video board. We also introduce an exercise that uses the OLED.

A. SPI controller specification

The RVfpga System's SPI module is from OpenCores (https://opencores.org/projects/simple_spi). If you download the package, a document is provided that describes the high-level specification of the module. This document is also provided here:

*[RVfpgaEL2Nexys
VideoNoDDRPath]/src/VeeRwolf/Peripherals/spi/docs/simple_spi.pdf*

We summarize the main operation and features of the SPI module; however, refer to the above document for additional information.

This module has the following main features:

- It is compatible with Motorola's SPI specifications
- It uses the 8-bit WISHBONE RevB.3 classic interface
- It contains a 4-entry read FIFO buffer and a 4-entry write FIFO buffer
- It allows interrupt generation after 1, 2, 3, or 4 transferred bytes
- It can operate with a wide range of input clock frequencies
- It is fully synthesizable

Section 3 of the SPI core specification describes the control and status registers available inside the SPI module, each of which is assigned to a different address (see Table 1). The base address of the SPI controller is **0x80001100**. These registers are described in detail below.

Table 1. SPI Registers

Name	Address	Width	Access	Description
SPCR	0x80001100	8	R/W	Control register
SPSR	0x80001108	8	R/W	Status register
SPDR	0x80001110	8	R/W	Data register
SPER	0x80001118	8	R/W	Extensions register
SPCS	0x80001120	8	R/W	CS register

The SPI Control Register (SPCR) controls the SPI module; Table 2 shows the function of each of its bits.

Table 2. SPCR bits

Bit	Access	Name & Description
0:1	R/W	SPR SPI clock Rate: These bits select the SPI clock rate.
2	R/W	CPHA Clock Phase: Determines the phase of sampling and sending data. When CPHA = 1, new data is shifted onto the wire at the leading edge and data is sampled on the trailing edge. When CPHA = 0, new data is shifted onto the wire at the trailing edge and sampled on the leading edge.
3	R/W	CPOL Clock Polarity: Determines idle state of SPI clock (SCK). When CPOL = 0, SCK idles at 0, when CPOL = 1, SCK idles at 1.
4	R/W	MSTR Mode Select: When MSTR = 1, the SPI core is a controller device. This is the only supported mode for this controller.
6	R/W	SPE SPI Enable: When SPE = 1, the SPI core is enabled. When it is cleared (SPE = 0), the SPI core is disabled.
7	R/W	SPIE

		SPI Interrupt Enable: When SPIE = 1, when the SPI Interrupt Flag in the status register is set, the host is interrupted.
--	--	--

The SPI Status Register (SPSR) provides the status of the SPI module; Table 3 shows the function of each of its bits.

Table 3. SPSR bits

Bit	Access	Description
0	R/W	RFEMPTY Read FIFO Empty: If RFEMPTY = 1, the read FIFO is empty.
1	R/W	RFFULL Read FIFO Full: If RFFULL = 1, the read FIFO is full.
2	R/W	WFEMPTY Write FIFO Empty: IF WFEMPTY = 1, the write FIFO is empty.
3	R/W	WFFULL Write FIFO Full: IF WFFULL = 1, the write FIFO is full.
6	R/W	WCOL Write Collision flag: When WCOL = 1, the SPDATA register was written to while the Write FIFO was full. Writing a 1 to WCOL clears this bit.
7	R/W	SPIF SPI Interrupt Flag: SPIF = 1 upon completion of a transfer block. If SPIF is asserted ('1') and SPIE is set, an interrupt is generated. Writing a 1 to SPIF clears it.

The SPI Data Register (SPDR) provides the data to read or write. The SPI controller includes 4 x 8-bit Write Buffer and a 4x 8-bit Read Buffer.

The SPI Extended Register (SPER) provides some additional functionality; Table 4 describes the different fields that it contains.

Table 4. SPER bits

Bit	Access	Description
0:1	R/W	ESPR Extended SPI Clock Rate Select: Add two bits to the SPR (SPI Clock Rate Select).
6:7	R/W	ICNT Interrupt Count: Determine the transfer block size. The SPIF bit is set after ICNT transfers. Thus, it is possible to reduce kernel overhead due to reduced interrupt service calls.

Finally, the SPI Chip Select (SPCS) register selects which peripheral to use. The width of this signal is configurable through parameter SS_WIDTH (SPI Select Width). In the RVfpga System, only one peripheral exists for each SPI interface, so SS_WIDTH = 1.

TASK: Locate the declaration of registers SPCR, SPSR, SPDR, SPER and SPCS in the SPI module, as well as the definition of their addresses. The SPI module is available inside folder *[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Peripherals/spi*.

B. UG-2832HSWEG04 OLED specification

The Nexys Video board includes an UG-2832HSWEG04 OLED. You can find the complete information for the device in its data sheet, located here:

[SAS1-B020-B](#) and [SSD1780](#)

The UG-2832HSWEG04 is a 0.91-inch monochrome passive-matrix OLED display module with a resolution of 128 × 32 pixels, driven by the SSD1306 controller IC. It features a 1/32 duty cycle and offers high contrast (>2000:1) and wide viewing angles (>160°). The module operates with a logic supply voltage of 1.65–3.3 V and a display driving voltage of 7.0–7.5 V, consuming only a few milliamps during normal operation.

The display's pixels are organized in a 128-segment by 32-common matrix, and image data is written through a 4-wire SPI serial interface consisting of the signals CS#, RES#, D/C#, SCLK, and SDIN. The controller includes several command and data registers that configure display parameters such as multiplex ratio, contrast level, segment remap, and COM scan direction. Communication with the device is performed by transmitting a command or data byte following the SPI protocol, where the D/C# line determines the data type.

The UG-2832HSWEG04 can be powered either with an external VCC or through its internal charge-pump DC/DC converter, depending on the system design. A recommended initialization sequence includes setting the charge pump, multiplex ratio, display offset, and VCOMH level before enabling the display. The typical connection between an FPGA and the display module is illustrated in Figure 5.

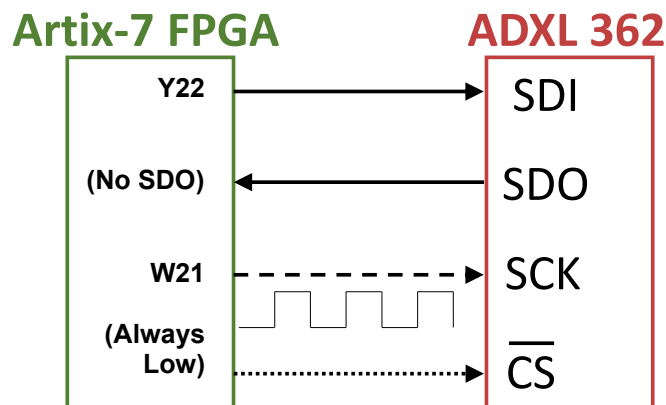


Figure 5. UG-2832HSWEG04 OLED interface with the Nexys Video board

The communication protocol and command structure of the UG-2832HSWEG04 are relatively complex. Detailed information regarding command sequences, initialization parameters, and data addressing modes can be found in [SSD1780](#).

4. LOW-LEVEL IMPLEMENTATION

A. SPI OLED low-level implementation

In the first part of this lab, we showed how to use the RVfpga System's SPI modules, and in this last part of the lab we describe how the SPI module is implemented in RVfpga. Similar to the format from previous labs, we divide the analysis of the SPI controller into three phases:

1. Physical connection between the SoC and the OLED (left shadowed region in Figure 6)
2. Integration of the SPI controller, which is included inside the VeeRwolfX System Controller (middle shadowed region in Figure 6)
3. Connection between the SPI controller and the VeeR EL2 Core (right shadowed region in Figure 6)

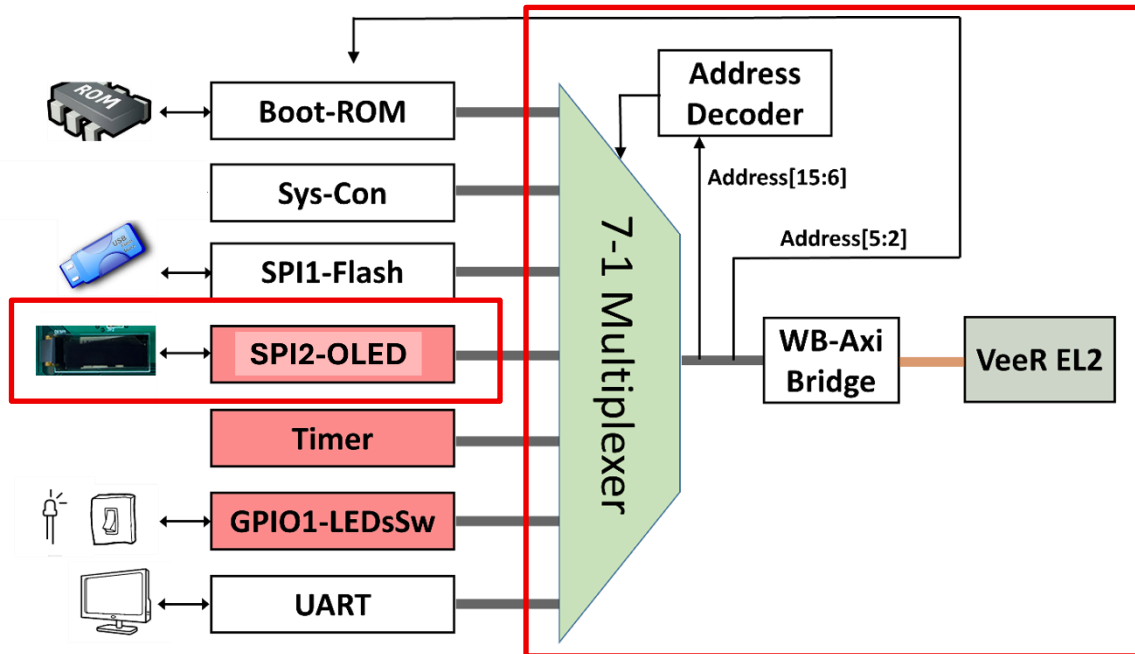


Figure 6. SPI controller integrated into the RVfpga System

1. Physical connection of the OLED and the SoC

As with other peripherals, the RVfpgaEL2-Nexys Video constraints file must include the physical connections to the OLED display.

The constraints file of the project ([RVfpgaEL2_NexysVideo-NoDDRPath]/src/rvfpganexys.xdc) defines the connection between the SoC's input/output signals and the board's device pins.

Only two signal lines are used for data communication, since on the Nexys Video board the OLED's CS line is permanently pulled low—meaning the device is always enabled—and the module operates in 3-wire SPI mode, which does not include an SDO line.

The remaining four lines are specific control signals dedicated to this OLED module:

o_oled_dc - selects between Data and Command mode

o_oled_res - controls the reset function

o_oled_vbat - controls the internal power supply

o_oled_vdd - controls the logic power

All four control signals are active-low.

These connections are defined in the constraints file as shown in Figure 9.


```

63 ## OLED Display
64 set_property -dict { PACKAGE_PIN W22 IOSTANDARD LVCMOS33 } [get_ports { o_oled_dc }]; #IO_L7N_T1_D10_14 Sch=oled_dc
65 set_property -dict { PACKAGE_PIN U21 IOSTANDARD LVCMOS33 } [get_ports { o_oled_res }]; #IO_L4N_T0_D05_14 Sch=oled_res
66 set_property -dict { PACKAGE_PIN W21 IOSTANDARD LVCMOS33 } [get_ports { o_oled_sclk }]; #IO_L7P_T1_D09_14 Sch=oled_sclk
67 set_property -dict { PACKAGE_PIN Y22 IOSTANDARD LVCMOS33 } [get_ports { o_oled_sdin }]; #IO_L9N_T1_DQ5_D13_14 Sch=oled_sdin
68 set_property -dict { PACKAGE_PIN P20 IOSTANDARD LVCMOS33 } [get_ports { o_oled_vbat }]; #IO_0_14 Sch=oled_vbat
69 set_property -dict { PACKAGE_PIN V22 IOSTANDARD LVCMOS33 } [get_ports { o_oled_vdd }]; #IO_L3N_T0_DQ5_EMCLK_14 Sch=oled_vdd

```

Figure 7. Connection of the SoC and the OLED (file *rvfpaganexys.xdc*).

In lines 39 to 44 of the top-module of RVfpgaEL2-Nexys Video (i.e., the **rvfpagaNexys Video** module) you can see these two signals connected to the SoC (left part of Figure 8), and the end of that module are their connection with the **veerwolf_core** module (right part of Figure 8).

<pre> 25 module rvfpaganexys 26 #(parameter bootrom_file = "boot_main.mem") 27 (input wire clk, 28 29 output wire o_flash_cs_n, 30 output wire o_flash_mosi, 31 input wire i_flash_miso, 32 33 input wire i_uart_rx, 34 output wire o_uart_tx, 35 36 inout wire [7:0] i_sw, 37 output reg [7:0] o_led, 38 39 output wire o_oled_sclk, 40 output wire o_oled_sdin, 41 output wire o_oled_dc, 42 output wire o_oled_res, 43 output wire o_oled_vbat, 44 output wire o_oled_vdd 45); </pre>	<pre> 257 .i_ram_rid (ram_rid), 258 .i_ram_rdata (ram_rdata), 259 .i_ram_rresp (ram_rresp), 260 .i_ram_rlast (ram_rlast), 261 .i_ram_rvalid (ram_rvalid), 262 .o_ram_rready (ram_rready), 263 .i_ram_init_done (1'b1), 264 .i_ram_init_error (1'b0), 265 .io_data ({i_sw_slice[15:0]}, 266 .o_oled_sclk(o_oled_sclk), 267 .o_oled_mosi(o_oled_sdin), 268 .o_oled_dc(o_oled_dc), 269 .o_oled_res(o_oled_res), 270 .o_oled_vbat(o_oled_vbat), 271 .o_oled_vdd_o(o_oled_vdd) 272); </pre>
--	--

Figure 8. Connection of the OLED with the top-module (file *rvfpaganexys.sv*).

TASKS: Follow these four signals (*o_oled_cs_n*, *o_oled_mosi*, *i_oled_miso* and *oled_sclk*) from the constraints file to the VeeRwolfX SoC module. You will need to inspect the following files:

```

[RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpaganexys.xdc
[RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpaganexys.sv
[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/veerwolf_core.v

```

2. OLED_SPI core implementation

Due to the presence of four additional control lines, a specialized SPI core based on *simple_spi*, named *oled_spi*, is used in this design.

An 8-bit register is added at the address 0x80001128 to independently control these four signals.

The *oled_spi* core integrates an instance of *simple_spi* and includes additional logic to handle access to this new register address.

This implementation is illustrated in figure 9.

```

45 wire wb_acc = cyc_i & stb_i; // WISHBONE access
46 wire wb_wr = wb_acc & we_i; // WISHBONE write access
47
48 always @(posedge clk_i)
49   if (rst_i)
50     begin
51       oled_control_lines <= 4'hF; // all control lines are active low
52     end
53   else if (wb_wr)
54     begin
55       if (adr_i == 3'b101)
56         begin
57           oled_control_lines <= dat_i[3:0];
58         end
59     end
60
61 reg [3:0] oled_control_lines;
62 reg [7:0] oled_data_o;
63
64 always @(posedge clk_i)
65   case (adr_i)
66     3'b101: oled_data_o = {4'b0000, oled_control_lines};
67     default: oled_data_o = 8'h00;
68   endcase
69
70 assign dat_o = oled_data_o | spi_data_o;
71
72 assign oled_dc_o = oled_control_lines[0];
73 assign oled_res_o = oled_control_lines[1];
74 assign oled_vbat_o = oled_control_lines[2];
75 assign oled_vdd_o = oled_control_lines[3];

```

Figure 9. Extra address access handling (file oled_spi.v)

3. Integration of the SPI2-OLED module in the SoC

At lines 387-403 of module **veerwolf_core** (*[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/veerwolf_core.v*) the SPI module for the OLED is instantiated (see Figure 10).

```

451 // SPI for the Accelerometer
452 wire [7:0] spi2_rdt;
453 assign wb_s2m_spi_oled_dat = {24'd0, spi2_rdt};
454 wire spi2_irq;
455
456 oled_spi spi2
457   // Wishbone slave interface
458   .clk_i (clk),
459   .rst_i (wb_rst),
460   .adr_i (wb_m2s_spi_oled_adr[2] ? 3'd0 : wb_m2s_spi_oled_adr[5:3]),
461   .dat_i (wb_m2s_spi_oled_dat[7:0]),
462   .we_i (wb_m2s_spi_oled_we),
463   .cyc_i (wb_m2s_spi_oled_cyc),
464   .stb_i (wb_m2s_spi_oled_stb),
465   .dat_o (spi2_rdt),
466   .ack_o (wb_s2m_spi_oled_ack),
467   .inta_o (spi2_irq),
468   // SPI interface
469   .sck_o (o_oled_sclk),
470   .mosi_o (o_oled_mosi),
471   // OLED control lines
472   .oled_dc_o(o_oled_dc),
473   .oled_res_o(o_oled_res),
474   .oled_vbat_o(o_oled_vbat),
475   .oled_vdd_o(o_oled_vdd_o)
476   );

```

Figure 10. Integration of the SPI2-OLED module (file veerwolf_core.v).

As usual with peripherals, the interface of the module can be divided into two blocks: Wishbone signals (Table 5) and external I/O signals (Table 6). The Wishbone signals enable the VeeR EL2 Core to communicate with the ADC using the SPI protocol.

Table 5. Wishbone Signals

Port	Width	Direction	Description
cyc_i	1	Inputs	Indicates valid bus cycle (core select)
adr_i	15	Inputs	Address inputs
dat_i	32	Inputs	Data inputs

dat_o	32	Outputs	Data outputs
sel_i	4	Inputs	Indicates valid bytes on data bus (during valid cycle it must be 0xf)
ack_o	1	Output	Acknowledgment output (indicates normal transaction termination)
err_o	1	Output	Error acknowledgment output (indicates an abnormal transaction termination)
rty_o	1	Output	Not used
we_i	1	Input	Write transaction when asserted high
stb_i	1	Input	Indicates valid data transfer cycle
inta_o	1	Output	Interrupt output

Table 6. External I/O Signals

Port	Width	Direction	Description
miso_i	1	Input	Controller data Input - Peripheral data Output
mosi_o	1	Output	Controller data Output - Peripheral data Input
ss_o	1	Output	Chip Select
sck_o	1	Output	System clock

As shown in Figure 10, bits [5:2] of the address provided by the core in the Wishbone bus signal (*wb_m2s_spi_oled_adr[5:2]*) are used for selecting one among the 5 available SPI registers (Table 1).

4. Connection between the SPI Controller and the VeeR EL2 Core

As explained in previous labs, the device controllers are connected to the VeeR EL2 Core through a multiplexer and a bridge (Figure 6). Remember that the 7:1 multiplexer is instantiated in file *[RVfpgaEL2NexysVideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb_intercon.v*. Then, the *wb_intercon* module is instantiated in file *[RVfpgaEL2NexysVideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb_intercon.vh*. This latter file is included in the *veerwolf_core* module located here: *[RVfpgaEL2NexysVideoNoDDRPath]/src/VeeRwolf/veerwolf_core.v*.

The multiplexer selects which peripheral to read or write, connecting the CPU (*wb_io_** signals) with the Wishbone Bus of one peripheral, depending on the address. For example, if the address generated by the CPU is in the range 0x80001100-0x8000113F, the OLED module is selected, and thus signals *wb_io_** are connected to signals *wb_spi_oled_**.

```

104 ~ wb_mux
105 ~ #(.num_slaves (7),
106   .MATCH_ADDR ({32'h00000000, 32'h00001000, 32'h00001040, 32'h00001100, 32'h00001200, 32'h00001400, 32'h00002000}),
107   .MATCH_MASK ({32'hffffff00, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffff00}))
108 ~ wb_mux_io
109 ~ (.wb_clk_i (wb_clk_i),
110   .wb_rst_i (wb_rst_i),
111   .wbm_adr_i (wb_io_adr_i),
112   .wbm_dat_i (wb_io_dat_i),
113   .wbm_sel_i (wb_io_sel_i),
114   .wbm_we_i (wb_io_we_i),
115   .wbm_cyc_i (wb_io_cyc_i),
116   .wbm_stb_i (wb_io_stb_i),
117   .wbm_cti_i (wb_io_cti_i),
118   .wbm_bte_i (wb_io_bte_i),
119   .wbm_dat_o (wb_io_dat_o),
120   .wbm_ack_o (wb_io_ack_o),
121   .wbm_err_o (wb_io_err_o),
122   .wbm_rty_o (wb_io_rty_o),
123   .wbs_adr_o ({wb_rom_adr_o, wb_sys_adr_o, wb_spi_flash_adr_o, wb_spi_oled_adr_o, wb_ptc_adr_o, wb_gpio_adr_o, wb_uart_adr_o}),
124   .wbs_dat_o ({wb_rom_dat_o, wb_sys_dat_o, wb_spi_flash_dat_o, wb_spi_oled_dat_o, wb_ptc_dat_o, wb_gpio_dat_o, wb_uart_dat_o}),
125   .wbs_sel_o ({wb_rom_sel_o, wb_sys_sel_o, wb_spi_flash_sel_o, wb_spi_oled_sel_o, wb_ptc_sel_o, wb_gpio_sel_o, wb_uart_sel_o}),
126   .wbs_we_o ({wb_rom_we_o, wb_sys_we_o, wb_spi_flash_we_o, wb_spi_oled_we_o, wb_ptc_we_o, wb_gpio_we_o, wb_uart_we_o}),
127   .wbs_cyc_o ({wb_rom_cyc_o, wb_sys_cyc_o, wb_spi_flash_cyc_o, wb_spi_oled_cyc_o, wb_ptc_cyc_o, wb_gpio_cyc_o, wb_uart_cyc_o}),
128   .wbs_stb_o ({wb_rom_stb_o, wb_sys_stb_o, wb_spi_flash_stb_o, wb_spi_oled_stb_o, wb_ptc_stb_o, wb_gpio_stb_o, wb_uart_stb_o}),
129   .wbs_cti_o ({wb_rom_cti_o, wb_sys_cti_o, wb_spi_flash_cti_o, wb_spi_oled_cti_o, wb_ptc_cti_o, wb_gpio_cti_o, wb_uart_cti_o}),
130   .wbs_bte_o ({wb_rom_bte_o, wb_sys_bte_o, wb_spi_flash_bte_o, wb_spi_oled_bte_o, wb_ptc_bte_o, wb_gpio_bte_o, wb_uart_bte_o}),
131   .wbs_dat_i ({wb_rom_dat_i, wb_sys_dat_i, wb_spi_flash_dat_i, wb_spi_oled_dat_i, wb_ptc_dat_i, wb_gpio_dat_i, wb_uart_dat_i}),
132   .wbs_ack_i ({wb_rom_ack_i, wb_sys_ack_i, wb_spi_flash_ack_i, wb_spi_oled_ack_i, wb_ptc_ack_i, wb_gpio_ack_i, wb_uart_ack_i}),
133   .wbs_err_i ({wb_rom_err_i, wb_sys_err_i, wb_spi_flash_err_i, wb_spi_oled_err_i, wb_ptc_err_i, wb_gpio_err_i, wb_uart_err_i}),
134   .wbs_rty_i ({wb_rom_rty_i, wb_sys_rty_i, wb_spi_flash_rty_i, wb_spi_oled_rty_i, wb_ptc_rty_i, wb_gpio_rty_i, wb_uart_rty_i});

```

**Figure 11. 7-1 multiplexer selects the peripheral to connect to the CPU
(wb_intercon.v).**

5. EXERCISES

Exercise 1. Create a RISC-V assembly program that reads the eight most significant bits of the X-axis, Y-axis, and Z-axis olederation data and then displays those values on the 8-digit 7-Segment Displays.

Exercise 2. Another common serial communication protocol is called I2C (pronounced “eye two see” or also “eye squared see”). The temperature sensor on the Nexys Video board uses this protocol. Expand the RVfpga System to include an I2C controller, and connect it with the Nexys Video board’s ADT7420 temperature sensor (<https://www.analog.com/media/en/technical-documentation/data-sheets/adt7420.pdf>). Then write a program that communicates with this new peripheral and displays the temperature on the 7-segment displays.